

(TR-104) PYTHON WITH AI/ML

Training Day 50 Report

Date: 20 March 2026

Topic Covered: AI Consumption Tools, RAG Pipeline Optimization & Streamlit Integration

The fiftieth day of training focused on exploring different AI-powered tools used across various domains and optimizing the RAG-based AI Assistant project for improved response time and frontend integration using Streamlit.

- Explored multiple AI tools used for development, content creation, productivity, and media generation.
- Learned about MIT App Inventor for AI-based mobile application development.
- Explored Teachable Machine for building simple machine learning models without coding.
- Reviewed Grammarly and understood AI-assisted grammar and writing improvement systems.
- Explored Soundraw for AI-generated music creation workflows.
- Studied Pictory for converting content into AI-generated videos.
- Optimized the response time of the RAG-based AI Assistant project.
- Replaced the single llama3.2 model with three specialized qwen2.5 models for improved CPU performance and faster inference.
- Added query detection logic to skip unnecessary rewrite model calls when the query is already correctly written.
- Optimized guardrail prompts to make them shorter and more efficient for lightweight models.
- Separated rewrite, RAG, and guardrail functionalities into independent model instances using app.state.
- Added warmup calls during server startup to reduce first-response latency.
- Built a Streamlit frontend connected with the existing FastAPI RAG backend.
- Implemented chat interface with session-based conversation history management.
- Added backend status checking functionality inside the Streamlit sidebar.

- Displayed API response time using the X-Response-Time header.
- Handled multiple API response cases including successful response, rate limiting (429), server errors (500), connection issues, and timeout handling.
- Fixed 422 Unprocessable Entity error caused by incorrect request body field mapping between frontend and backend.
- Successfully tested complete end-to-end communication between Streamlit frontend, FastAPI backend, and Ollama models.
- Strengthened understanding of integrating frontend interfaces with AI-powered backend systems.
- **GitHub Link:**
https://github.com/JaspinderKaurWalia26/rag_pipeline
- **Blog Link:**
<https://medium.com/@jaspinderkaurwalia855/how-i-built-a-chat-ui-for-my-rag-api-using-streamlit-6d0637895664>